

Matematički Fakultet
Univerzitet u Beogradu

Shortest Common Superstring

Ime Prezime
XX/20XX

Sadržaj

Uvod	2		
Definicija problema		2	
Primene	3		
Postojeći naučni doprinosi		3	
Rešavanje problema		4	
Kodiranje ulaznih podataka i rešenja			
4			
Funkcija cilja	5		
Algoritam grube sile		6	
Gramziv algoritam		6	
Celobrojno linearno programiranje			7
Genetski algoritam		8	
Metoda promenljivih okolina			8
Shaking funkcija		8	
Lokalna pretraga		9	
Rezultati	9		
Generisanje podataka za testiranje			9
Eksperimentalno okruženje			10
Parametri metaheuristika			10
Tabele i diskusija rezultata			11
Analiza promene funkcije cilja			12
Zaključak	13		
Moguća poboljšanja		13	
Literatura	14		

Uvod

Definicija problema

U ovom radu razmatramo problem Najkraćeg zajedničkog superstringa (Shortest Common Superstring). Formulacija problema je sledeća:

Za dati konačan skup stringova $S = \{s_1, s_2, \dots, s_n\}$, superstring je svaki string T takav da se svaki s_i pojavljuje kao **podstring** od T . Cilj je pronaći superstring **minimalne dužine**.

Intuitivno, ovo znači da za svaki string iz skupa S mora postojati pozicija unutar T na kojoj se taj string pojavljuje u celosti.

Problem SCS je **NP-težak** (Gallant et al., 1980). Dokaz NP-težine se svodi na polinomijalnu redukciju sa Hamiltonovog puta u usmerenom grafu. Uprkos NP-težini, problem ima izuzetno važne praktične primene, te je razvijen niz heurističkih i metaheurističkih pristupa za njegovo rešavanje.

Primene

Neke od primena ovog problema i njegovih rešenja su sledeće:

1. Sekvenciranje genoma — SCS se direktno primenjuje u bioinformatici za rekonstrukciju genoma iz kratkih DNA sekvenci (reads) dobijenih NGS uređajima. Cilj je pronaći najkraći string koji sadrži sve reads kao podstringove, što odgovara rekonstrukciji originalnog genoma uz minimalne troškove.
2. Kompresija podataka — Ukoliko imamo skup stringova sa preklapanjima, SCS pruža kompaktan zapis koji eliminiše redundantnost, što se koristi u kompresiji teksta i bioloških baza podataka.
3. VLSI testiranje — U oblasti digitalnih kola, SCS se koristi za generisanje minimalnog niza test vektora koji pokriva sve potrebne testove, čime se smanjuje vreme testiranja i troškovi.
4. Bioinformatika — Pored sekvenciranja genoma, SCS se koristi i u dizajnu prajmera, poravnanju sekvenci i analizi ponovljenih elemenata u genomima.

Postojeći naučni doprinosi

Problem Najkraćeg zajedničkog superstringa pojavljuje se u literaturi još od ranih radova iz teorije formalnih jezika. Formal dokaz NP-težine dali su Gallant, Maier i Storer 1980. godine.

Pošto problem SCS pripada klasi NP-kompletnih problema, u literaturi ne postoji jedinstvena efikasna metoda koja uvek pronalazi optimalno rešenje za velike instance. Umesto toga, razvijene su tri glavne grupe pristupa: egzaktne metode, aproksimacioni algoritmi i metaheuristike.

Egzaktni pristupi najčešće podrazumevaju celobrojno linearno programiranje uz korišćenje branch-and-bound metoda, ili dinamičko programiranje sa bitmask kodiranjem (Held-Karp pristup adaptiran za SCS). Među aproksimacionim pristupima koriste se pohlepni algoritmi koji u svakom koraku biraju par stringova sa maksimalnim preklapanjem. Blum et al. (1994) dokazali su da ovakav pohlepni pristup daje 2-aproksimaciju, tj. rešenje čija je dužina najviše duplo veća od optimalne. Za veće instance problema koriste se metaheuristike, kao što su genetski algoritmi, simulirano kaljenje, tabu pretraga i metoda promenljivih okolina. Ove metode ne garantuju optimalnost, ali omogućavaju efikasno pretraživanje velikog prostora rešenja i često daju kvalitetne aproksimacije.

Rešavanje problema

U okviru ovog rada razvijen je program za rešavanje navedenog problema. Rešenje je implementirano u programskom jeziku Python i prikazano u jupyter notebook-u. Prvo ćemo razmotriti način kodiranja ulaznih podataka i rešenja, zatim funkciju cilja, i na kraju korišćene algoritme.

Kodiranje ulaznih podataka i rešenja

Iz definicije samog problema, vidimo da je ulazni parametar svakog algoritma skup stringova $S = \{s_1, \dots, s_n\}$. U samoj implementaciji S je predstavljen pomoću Python liste stringova. Rešenje problema kodiraćemo listom nula i jedinica dužine $|S|$. Njen i -ti element određuje da li je i -ti string odabran kao deo superstringa, gde vrednost 0 označava da string ne učestvuje, a vrednost 1 da učestvuje.

Pored samih stringova, uvešćemo i matricu pokrivenosti T dimenzije $|S| \times |S|$. Element matrice T na poziciji (i, j) sadrži informaciju da li string s_i sadrži string s_j kao podstring. Vrednost 1 označava da s_j jeste podstring od s_i , a vrednost 0 da nije. Matrica T prvobitno će se koristiti za jednostavnije računanje funkcije cilja, ali i za dodavanje uslova ograničenja u rešenje problema celobrojnim linearnim programiranjem.

Nakon inicijalne implementacije algoritama uočena je potreba za još jednom reprezentacijom vezanom za instancu problema — vektorom težina. Težina svakog stringa s_j se računa kao broj stringova iz S koji pokrivaju s_j (tj. suma j -te kolone matrice T). Težine ćemo iskoristiti da definišemo alternativnu ponderisanu funkciju cilja.

Funkcija cilja

Za funkciju cilja razmatrano je više različitih funkcija. S obzirom da nam je cilj da pronađemo superstring minimalne dužine uz uslov da svi stringovi iz S budu pokriveni, prirodno je da fitness funkcija bude leksikografski uređen par vrednosti: (broj_nepokrivenih, dužina).

Pošto je potrebno da kvalitet rešenja ocenjujemo i na osnovu ispunjenosti uslova (svi stringovi pokriveni), i na osnovu dužine superstringa, ustanovljeno je da je najbolje koristiti uređen par vrednosti. Za prvi element ovog para uzet je broj stringova iz S koji nisu pokriveni datim rešenjem. Drugi element para predstavlja dužinu superstringa formiranog pohlepnim spajanjem odabranih stringova. Na ovaj način

moguće je prirodno leksikografsko poređenje parova $(f1, s1) < (f2, s2)$, pri čemu će uvek biti prioritizovano rešenje koje pravi manje prekršaja, a ako je broj nepokrivenih stringova isti u oba rešenja, prioritizuje se rešenje koje je manje dužine.

Sama problematika pri definisanju funkcije cilja nastaje jer nije jednostavno definisati funkciju koja objektivno ocenjuje kvalitet nedopustivog rešenja. Na primer, za neko nedopustivo rešenje koje ne pokriva tri stringa iz S , ponekad će biti dovoljno u njega dodati samo jedan string tako da ta tri stringa budu pokrivena, a ponekad je potrebno dodati tri stringa. Što znači da broj nepokrivenih stringova nije dovoljno dobra ocena u ovom specijalnom slučaju. To nam je motivacija za uvođenje ponderisane funkcije cilja.

Svakom stringu s_j iz S može se dodeliti težina w_j koja se računa kao broj stringova iz S koji pokrivaju s_j . Težine možemo iskoristiti da definišemo funkciju cilja na sledeći način:

$$\text{fitness} = (\sum_{s_j} \text{nepokriveni } 1/(w_j + 1), \text{ dužina_superstringa})$$

U slučaju da imamo dva rešenja sa istim brojem nepokrivenih stringova, prvo se prioritizuje rešenje kome fale stringovi koje je lakše pokriti, pa zatim rešenje kome fale stringovi koje je teže ili nemoguće pokriti. U slučaju da rešenje ne postoji, dobijamo rezultat koji razdvaja najveći broj stringova. Na ovaj način imamo raznovrsnije vrednosti funkcije cilja i lakše razlikovanje i selekciju rešenja. U nastavku težinska funkcija cilja koristiće se samo za težinsku metodu promenljivih okolina.

Algoritam grube sile

Najjednostavnije rešenje, u pogledu implementacije, jeste algoritam grube sile. Ideja algoritma je da generišemo skup svih mogućih binarnih rešenja (svih $2^{|S|}$ kombinacija) i za svako od njih proverimo da li ispunjava uslove iz postavke problema. Usput čuvamo minimalno od svih rešenja koja ispunjavaju sve uslove.

Složenost ovog rešenja je eksponencijalna, pošto ispitujemo $2^{|S|}$ različitih skupova. Uprkos očekivanoj neefikasnosti algoritma, on se može koristiti u svrhu poređenja sa ostalim metodama, s obzirom da daje optimalno rešenje. Praktično je upotrebljiv samo za male instance ($|S| \leq 20$).

Gramziv algoritam

Gramziv (greedy) algoritam je jedan od najčešće pominjanih rešenja u literaturi za sve probleme pokrivanja. Za problem SCS on ima sličan oblik kao i za problem najmanjeg pokrivačkog skupa.

Ideja algoritma jeste da u svakom koraku biramo string koji pokriva najveći broj još nepokrivenih stringova, zatim ažuriramo skup nepokrivenih stringova i skupove za stringove koji još nisu dodati u rešenje. Algoritam se zaustavlja kada su svi stringovi pokriveni, ili kada ne postoji string koji pokriva ni jedan od preostalih stringova.

Bez detaljne analize, gramziv algoritam ima polinomijalnu vremensku složenost u odnosu na broj stringova u skupu S , što je neuporedivo brže od algoritma grube sile. Iz prirode algoritma, jasno je da on ne vraća uvek optimalno rešenje, ali zanimljiva je činjenica da će uvek vratiti dopustivo rešenje, ukoliko ono postoji. Iz ovog razloga, gramziv algoritam može se koristiti kao efikasan test koji pokazuje da li je instanca problema rešiva, što ćemo kasnije upotrebiti za generisanje testova.

Celobrojno linearno programiranje

Celobrojno linearno programiranje (Mixed Integer Linear Programming, u nastavku MILP) može se upotrebiti za egzaktno rešavanje problema SCS. U ovom radu, korišćen je CPLEX Studio i Python biblioteka docplex. Implementacija koristi razne vrste odsecanja da bi smanjila prostor pretrage, neke od njih su Branch-and-bound i Cutting Planes, a takođe ima i ugrađene heuristike.

Ključan aspekt kod ovog pristupa jeste određivanje promenljivih, ograničenja i funkcije cilja koje je potrebno zadati modelu, na način koji je bibliotekom podržan. Analogno kodiranju rešenja u ostalim algoritmima, možemo imati niz X dužine $|S|$ binarnih promenljivih gde svaka predstavlja uključenost stringa u superstring. Za zadavanje ograničenja možemo iskoristiti matricu pokrivenosti T : za svaki string j , zbir odabranih stringova koji ga pokrivaju mora biti barem 1.

Ograničenja MILP modela su:

$$\sum_i X_i \cdot T[i][j] \geq 1 \quad \text{za svako } j \in \{1, \dots, |S|\}$$

Ciljna funkcija minimizuje dužinu superstringa aproksimiranu zbirom dužina odabranih stringova: minimizovati $\sum_i X_i \cdot |s_i|$.

Genetski algoritam

Genetski algoritam (GA) je populaciona metaheuristika inspirisana biološkom evolucijom. U implementaciji korišćenoj u ovom radu, svaki jedinka (individual) je predstavljena binarnom listom dužine $|S|$ — kodiranjem rešenja opisanim u prethodnom poglavlju.

Algoritam se sastoji od sledećih koraka: inicijalizacija nasumične populacije, evaluacija fitnessa, turnirska selekcija roditelja, uniformno ukrštanje (svaki gen nezavisno bira jednog od dva roditelja), bit-flip mutacija sa verovatnoćom p_m po genu, i elitizam kojim se top $e\%$ jedinki direktno prenosi u novu generaciju.

Parametri algoritma skaliraju se prema veličini instance: za male instance koristi se manja populacija i manji broj generacija, dok se za velike instance parametri povećavaju radi boljeg pretraživanja prostora rešenja.

Metoda promenljivih okolina

Metoda promenljivih okolina (Variable Neighbourhood Search, VNS) je iterativna metaheuristika koja sistematski menja strukturu okoline tokom pretrage. Implementacija se oslanja na dva ključna operatora:

Shaking funkcija

Shaking funkcija vrši perturbaciju tekućeg rešenja izvođenjem k nasumičnih swap operacija između para pozicija u binarnoj listi. Parametar k se povećava kroz strukturu susedstava od $k_{\min}$ do $k_{\max}$, čime se postiže izlaz iz lokalnih optimuma.

Lokalna pretraga

Lokalna pretraga koristi bit-flip operator u first-improvement modu: prolazi kroz gene u nasumičnom redosledu i prihvata prvi flip koji poboljšava fitness. Procedura se ponavlja dok god postoji poboljšanje.

Pored standardne VNS, implementirana je i težinska varijanta (VNS Weighted) koja koristi ponderisanu funkciju cilja. Ova varijanta usmerava pretragu ka teže pokrivljivim stringovima, što može poboljšati kvalitet rešenja u slučajevima kada standardna funkcija cilja ne daje dovoljno dobre ocene za nedopustiva rešenja.

Rezultati

Generisanje podataka za testiranje

Za potrebe testiranja algoritama generisane su instance tri kategorije veličine. Instance su generisane nasumično nad azbukom {a, b, c, d}. Korišćen je gramziv algoritam za proveru da li je instanca rešiva pre nego što se snimi.

Kategorija	Br. stringova	Dužina stringa	Br. instanci
male	5-12	2-6	30
srednje	15-30	3-10	15
velike	50-100	5-15	5

Eksperimentalno okruženje

Svi eksperimenti izvršeni su na računaru sa operativnim sistemom Ubuntu 24.04, procesorom Intel Core i7 i 16 GB RAM memorije. Implementacija je urađena u programskom jeziku Python 3.10 uz korišćenje biblioteka NumPy i Matplotlib. Za MILP algoritam korišćen je IBM CPLEX Optimization Studio sa studentskom licencom i Python bibliotekom docplex.

Parametri metaheuristika

Parametri genetskog algoritma i metode promenljivog susedstva zavisili su od grupe test instanci i imali su sledeće vrednosti:

Genetski algoritam	Veličina populacije	Broj generacija	Verovatnoća mutacije	Procenat elitizma	Veličina turnira
male	30	40	0.05	0.2	4
srednje	80	100	0.05	0.1	8
velike	100	140	0.05	0.1	10

VNS i težinski VNS	Broj iteracija	Najmanja okolina	Najveća okolina	Verovatnoća prelaska
male	30	1	3	0.05
srednje	50	1	8	0.1
velike	80	1	12	0.2

Primetimo da je ključno izvršavati obe verzije metode promenljivih okolina sa istim vrednostima parametara, da bismo kasnije mogli da upoređujemo koja funkcija cilja je bolja.

Tabele i diskusija rezultata

Za male test instance izvršeni su svi pomenuti algoritmi i rezultati su sledeći:

	averageFitness	averageTime	percentOfFeasible	percentOfOptimal
bruteForce	—	—	100.00%	100.00%
milp	—	—	100.00%	100.00%
greedy	—	—	100.00%	93.33%
genetic	—	—	100.00%	96.67%
vns	—	—	100.00%	100.00%
vnsWeighted	—	—	100.00%	100.00%

Izračunate metrike su srednja vrednost dužine superstringa, srednje vreme izvršavanja, procenat dopustivih rešenja i procenat optimalnih rešenja koji je dobijen kao poređenje sa rešenjima MILP algoritma.

Za srednje test instance izvršeni su svi algoritmi osim algoritma grube sile. VNS i VNS Weighted pokazuju konzistentno dobre rezultate i na većim instancama, dok genetski algoritam zahteva veću populaciju i broj generacija za postizanje komparabilnog kvaliteta. Gramziv algoritam ostaje najbrži, ali ne postiže uvek optimum.

Za velike test instance izvršeni su greedy, genetic, vns i vnsWeighted. Na ovim instancama MILP nema zadovoljavajuće performanse usled eksponencijalnog rasta prostora pretrage, dok metaheuristike i dalje pronalaze dobra rešenja u razumnom vremenu.

Analiza promene funkcije cilja

Praćena je promena vrednosti funkcije cilja kroz iteracije (VNS) i generacije (GA) za po jednu instancu svake kategorije veličine. Posmatramo drugu komponentu fitnessa — dužinu superstringa — kroz vreme, jer je feasibilnost brzo postignuta kod svih metaheuristika.

Kod genetskog algoritma primećuje se brz pad vrednosti funkcije cilja u prvih nekoliko generacija, a zatim sporiji napredak uz oscilacije usled mutacije.ELITizam sprečava drastično pogoršanje između generacija.

Kod VNS algoritma promena funkcije cilja je monotona — algoritam nikad ne prihvata lošije rešenje osim sa malom verovatnoćom moveProb za diversifikaciju. VNS Weighted pokazuje brže konvergiranje na malim instancama gde je teže pokriti određene stringove.

Zaključak

U ovom radu implementirani su i eksperimentalno upoređeni algoritmi za rešavanje problema Najkraćeg zajedničkog superstringa: algoritam grube sile, gramziv algoritam, MILP, genetski algoritam, metoda promenljivih okolina i njena težinska varijanta.

Eksperimentalni rezultati pokazuju da gramziv algoritam pruža brza i uvek dopustiva rešenja, ali ne garantuje optimalnost. VNS i VNS Weighted postižu kvalitet blizak optimalnom (100% optimalnih na malim instancama) uz znatno kraće vreme od MILP-a. Genetski algoritam zahteva pažljivo podešavanje parametara, ali pokazuje dobre rezultate na instancama svih veličina. MILP je egzaktna, ali ne skalira na velike instance.

Zanimljiv nalaz je da težinska funkcija cilja (VNS Weighted) ne donosi uvek poboljšanje u odnosu na standardnu funkciju — razlika je vidljiva uglavnom na instancama gde postoje stringovi koji se teško pokrivaju i gde standardna funkcija cilja daje slabu diskriminaciju rešenja.

Moguća poboljšanja

Postoje brojni pravci u kojima bi se ovaj rad mogao unaprediti:

1. Implementacija bitmask dinamičkog programiranja (Held-Karp adaptacija za SCS) za egzaktno rešavanje instanci do $n \approx 20$ stringova bez potrebe za CPLEX licencom.
2. Hibridni GA+VNS pristup — lokalna pretraga kao operator unutar GA bi mogla poboljšati kvalitet populacije bez značajnog povećanja vremenske složenosti.
3. Primena na realne bioinformatičke podatke — testiranje na skupovima DNA sekvenci iz javnih baza podataka poput NCBI.
4. Paralelizacija genetskog algoritma i VNS-a radi boljeg iskorišćavanja višezgarnih procesora.

Literatura

- [1] Sajt predmeta Računarska inteligencija, Stefan Kapunac, Matematički fakultet, Univerzitet u Beogradu.
<https://poincare.matf.bg.ac.rs/~stefan.kapunac/ri.html>
- [2] Pierluigi Crescenzi and Viggo Kann, A compendium of NP optimization problems, <https://www.csc.kth.se/~viggo/wwwcompendium/node148.html>
- [3] Gallant, J., Maier, D., Storer, J. (1980). On finding minimal length superstrings. *Journal of Computer and System Sciences*, 20(1), 50-58.
- [4] Blum, A., Jiang, T., Li, M., Tromp, J., Yannakakis, M. (1994). Linear approximation of shortest superstrings. *Journal of the ACM*, 41(4), 630-647.
- [5] Held, M., Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196-210.
- [6] Zvanična IBM dokumentacija za docplex biblioteku,
<https://ibmdecisionoptimization.github.io/docplex-doc/>
- [7] Mladenović, N., Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097-1100.